

System Architecture & Flow Guide

1. What is the Smart Email Inbox Triage Agent?

The Smart Email Inbox Triage Agent is a highly optimized Python-based automation system designed to act as an intelligent personal assistant for managing email clutter. It continuously monitors multiple Gmail accounts, analyzes incoming messages, and automatically flags important emails (like invoices, OTP security codes, contract deadlines, or human-to-human communications) to deliver instant notifications straight to Slack. It utilizes a two-layered triage system - first checking local rules, and falling back to a Google Gemini AI model - to maximize speed and eliminate unnecessary API costs.

2. Core Technologies & Tools Used

- * **Python 3.11+:** The foundational programming language used for building the agent's logic.
- * **Gmail API & Google OAuth2:** Establishes a secure, read-only token connection to scan Gmail folders without exposing passwords.
- * **Google Gemini API (google-genai SDK):** Processes text and uses advanced language models (gemini-2.0-flash) to evaluate email importance and generate action summaries.
- * **SQLite Database (Local):** Stores already processed email IDs locally (deduplication) so that no email is analyzed or notified about twice.
- * **Slack Webhooks & Block Kit:** Delivers beautifully formatted, color-coded, and highly readable notification cards to designated Slack channels.
- * **python-dotenv:** Enables safe, secure management of secret API keys, credentials, and configurations via a local .env file.

3. Step-by-Step Triage Workflow

Step 1: Fetching Unread Emails

The agent connects to the Gmail API and requests unread email stubs (IDs) from the INBOX and SPAM folders. By restricting this query to unread emails, it skips thousands of historic read emails, keeping scans fast.

Step 2: Local Deduplication Check

For each retrieved email, the agent checks its unique Gmail ID against the local SQLite database. If the ID is already marked as processed, it skips the email immediately. This guarantees that you never receive duplicate alerts.

Step 3: Fast, Free Pre-Filter Rules

Before making expensive AI calls, the email undergoes rule-based filtering. If the sender's domain matches your VIP list (like '@bank.com', '@workplace.com') or the subject/preview contains critical keywords (like 'OTP', 'Invoice', 'Deadline'), it is instantly flagged as 'Important' and bypasses Gemini AI entirely. This saves money and eliminates latency.

Step 4: AI Classification (Gemini)

If rules cannot decide, the agent sends the sender, subject, and snippet preview to Gemini. The AI analyzes it and returns a structured JSON object containing: Whether it is important, priority status (Normal/Urgent), and a concise 1-2 sentence action-oriented summary of the email's contents.

Step 5: Slack Alerts & Database Sync

If an email is determined to be important, the agent formats a premium notification card and sends it to Slack. Finally, the email

ID is saved to the SQLite database to avoid reprocessing.

4. Gemini API Free-Tier Rate Limit Solutions

Google's Gemini API Free Tier restricts usage to 15 Requests Per Minute (RPM) and 1,500 Requests Per Day (RPD). To ensure the agent runs stably and never crashes due to these limits, we implemented the following strategies:

- * Unread Filtering ('is:unread'):** We restricted Gmail API fetches to unread messages only. This prevents the agent from scanning older emails and ensures it only consumes API tokens for new, incoming mail.
- * Bypass Summarization for Rules:** VIP senders and keyword matches are marked important instantly. We deactivated the AI summarizer for these matches by default, utilizing the raw snippet instead. This reduces API consumption by up to 60%.
- * Safety Cap per Run (MAX_GEMINI_CALLS_PER_RUN = 15):** If you receive a massive spike of emails, the agent will process up to 15 emails using Gemini in a single run. Once this limit is hit, it exits cleanly and leaves the remaining emails unread. They will be naturally triaged during the next run.
- * Intelligent Backoff for 429 Errors:** If the API returns a '429 Resource Exhausted' rate limit error, the agent catches it and pauses for 30 seconds on the first retry, 60 seconds on the second, and 90 seconds on the third, allowing the quota bucket to reset.